SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – CLASS TEST

Course Code:	DSA05	**Course Title:**	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	**Assessment:**	Tool 3 – Class Test
Weightage:		**Total Marks:**	20 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL2 and BL4)			
Student Name:	P.Sunayana	**Reg. No.:**	192424303
Section / Batch:	Slot-A 2024	**Date of Submission:**	10-08-2026
Faculty In-charge:	**Dr. K. Veena Devi**	**Project Mode:**	Individual Submission

Code of Conduct

I ____P.Sunayana 192424303_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____P.Sunayana_____

Q1 (2 Marks)	Q2 (2 Marks)	Q3 (2 Marks)	Q4 (2 Marks)	Q5 (2 Marks)	Q6 (5 Marks)	Q7 (5 Marks)	Total (20 Marks)

CO2 - AT3 - CLASS TEST

Name:- P. Sunayana.

Reg.NO:- 192424303.

95

PART - A

1) Define a primary key and explain its purpose in a database table?

A primary key is a column (or) a combination of columns that uniquely identifies each record (row) in a database table.

purpose:-

1) Ensures every record is unique.
2) Doesn't allows null values.
3) Prevents duplicate values.
4) Helps establish relationships between tables.

Example:-

```
CREATE TABLE Student(
        Student_ID INT PRIMARY KEY,
        Name VARCHAR(50),
        Dept VARCHAR(50)
        );
```

Here Student_ID is uniquely identifies each student.

2) What is meant by CRUD operations in database management?

CRUD stands for the four basic operations performed on database records.

Operation	Meaning	SQL command
C	Create new records	INSERT
R	Read / retrieve records	SELECT
U	updating existing records	UPDATE
D	Delete records	DELETE

Example:-

```
INSERT INTO student values ( 1, 'John', 'CSE');
SELECT * from student;
UPDATE student SET Dept = 'IT' WHERE
    student_ID = 1;
DELETE FROM student WHERE student_ID = 1;
```

3) Differentiate between SQLite and MySQL.

SQLite:-

Type :- Embedded (bo) database.

Installation:- No separate server required.

Storage:- Usually stored in a single file.

Concurrency:- limited compared with MySQL.

Best suited for:- Small applications, testing, mobile/ desktop apps.

Administration:- Very simple.

MySQL:-

Type:- Client - server database.

Installation:- Required a database server

Storage:- User - server-managed database storage.

Concurrency:- Supports many simultaneous users.

Best suited for:- Web applications and large multi-user systems.

Administration:- Requires more configuration and administration.

4) What is a foreign key? Give an example.

A foreign key is a column in a table that refers to the primary key of another table.

→ It is used to establish a relationship between tables and maintain refuential integrity.

Example:-

```
CREATE TABLE Dept (
    Dept_ID INT PRIMARY KEY,
    Dept_Name VARCHAR(50)
    );

CREATE TABLE student(
    student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Dept_ID INT,
    FOREIGN KEY (Dept_ID) REFERENCES
    Dept(Dept_ID) );
```

Here, student.Dept_ID is a foreign key that refers Dept.Dept_ID.

5) Why is database normalization important?

Database normalization is the process of organizing data into tables to reduce data redundancy and improves data integrity.

Importance:-

1) Reduces duplicate data.
2) prevents data inconsistency.
3) Makes databases easier to maintain.
4) Improves data integrity.
5) Avoids, insertion, update, and deletion anomalies.
6) Make relationships between data clearer.

Example:-

Instead of storing the department name repeatedly for every student, we can create a separate Dept table and reference it using Dept_ID.

PART - B

95

6) University Database

Primary key and Foreign key

student (table name)

Student_ID	Name	Dept_ID
101	Arun	D01
102	Priya	D02
103	Ravi	D01

Department (table name)

Dept_ID	Dept_Name
D01	CSE
D02	IT

Relationship:-

The Dept_ID in the student table establishes a relationship with Dept_ID in the Department table.

This is a one to many relationship (1:M).

→ One dept can have many students.

→ Each student belongs to one department.

Primary key:-

→ Student_ID is the primary key of the student table.

→ Dept_ID is the primary key of the Dept table.

→ A primary key uniquely identifies each record and cannot contain NULL values.

Foreign key:-

→ Student_ID. Dept_ID is a foreign key.

→ It references Department. Dept_ID.

Example:-

```
CREATE TABLE Department(
    Dept_ID VARCHAR (10) PRIMARY KEY,
    Dept_Name VARCHAR(50);
    );
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Dept_ID VARCHAR(10),
FOREIGN KEY (Dept_ID)
    REFERENCES Department (Dept_ID)
    );
```

Conclusion:- Primary and foreign keys maintain accurate relationships and prevent inconsistent data between tables.

7) Online Shopping - Importance of Order details:-

An online shopping system contains..

* Customer.
* Product.
* Order.
* Order details.

An Order can contain multiple products, and the same product can appear in many different orders.

Example structure:-

Order

Order_ID	Customer_ID	OrderDate
1001	C01	10-05-2016

Product

Product_ID	Product_Name	Price
P01	Laptop	50,000
P02	Mouse	800

Order details

Order_ID	Product_ID	Quantity	Price
1001	P01	1	50,000
1001	P02	2	800

Problems if product information is stored directly in order:-

OrderID, CustomerID, Product1, Quantity1, Product 2, Quantity 2, Product 3, Quantity 3.

This may cause several problems:-

1) Data redundency.
2) Limited product.
3) Update anomaly.
4) Insertion anomaly.
5) Deletion anomaly.
6) Poor Scalability.
7) Normalization problems.

Conclusion:-

Orderdetails provides a flexible way to represent the many-to-many relationships between orders and products and avoids redundency and data anomalies.

8) Data Wrangling:

Dataset A

Student_ID	Name	Department	Marks
101	Arun	CSE	85
102	Priya	CSE	90
103	Ravi	IT	78.

Dataset B:-

Student_ID	Name	Marks
101	Arun	85
103	Ravi	78
104	Kumar	88.

The data-sets have different structures:-

Step 1:- Standardize the structure:-

Dataset B does not contain the Dept column. Add the missing Column:-

Student_ID	Name	Department	Marks
101	Arun	CSE	85
103	Ravi	IT	78
104	Kumar	unknown	88.

The dept of kumar is not available, so it should not be guessed.

Step 2:- Combine the datasets:-

Use a union/ concatination operation and remove duplicate records:-

Student_ID	Name	Department	Marks
101	Arun	CSE	85
102	Priya	CSE	90
103	Ravi	IT	78
104	kumar	unknown	88.

python Implementation:-

```
import pandas as pd
A = pd.DataFrame({
    'Student_ID': [101, 102, 103],
    'Name': ['Arun', 'Priya', 'Ravi'],
    'Department': ['CSE', 'CSE', 'IT'],
    'Marks': [85, 90, 78]})
B = pd.DataFrame({
    'Student_ID': [101, 103, 104],
    'Name': ['Arun', 'Ravi', 'kumar'],
    'Marks': [85, 78, 88]})
B['Department'] = 'unknown'
final = pd.concat([A, B], ignore_index=True)
final = final.drop_duplicates(subset='Student_ID')
print(final)
```

Output:-

	Student_ID	Name	Department	Marks
0	101	Arun	CSE	85
1	102	Priya	CSE	90
2	103	Ravi	IT	78
3	104	kumar	unknown	88.

Conclusion:-

The required data-Wrangling operations are schema alignment, concatenation, missing values are handled.

10) Data cleanup - Indian Mobile Numbers:-

Given data:- 9876543210
+91 - 9876543210
091 - 9876543210
9876543210
98 - 7654 - 3210.

cleaning steps:-

1) Remove spaces.
2) Remove hyphens.
3) Handle the +91 country code.
4) Handle the 091 prefix.
5) validate that the final number contains exactly 10 digits.
6) Indian mobile numbers should normally begin with 6, 7, 8 (or) 9.

python program:-

```
import re
numbers = [
    "9876543210",
    "+91 - 9876543210",
    "091 - 9876543210",
    "98765 - 43210",
    "98 - 7654 - 3210" ]
def clean_number(number):
    number = re.sub(r'[\s-]', '', number)
```

```
if (number . startswith ('+91'):
        number = number[3:]
    elif number . startswith ('091'):
        number = number[3:]
    if re.fullmatch (r'[6-9]\d{9}', number):
        return '+91 - ' + number
    return 'Invalid'
for number in numbers:
    print (number, "->", clean_number (number))
```

Output:-

```
9876543210          -> +91 - 9876543210
+91 - 9876543210    -> +91 - 9876543210
091 - 9876543210    -> +91 - 9876543210
98765 - 43210       -> +91 - 9876543210
98 - 7654 - 3210    -> +91 - 9876543210.
```

Regular Expression:-

```
[6-9]\d{9}
```

Means:-

→ [6-9] → first digit must be 6, 7, 8 (or) 9.

→ \d{9} → followed by (9) exactly 9 digits.

Conclusion:-

Regular expressions provide an efficient method for standardization telephone numbers and detecting invalid entries.

4) Data Exploration and Analysis using Pandas:-

The (dat) supermarket dataset contains:-

* Customer ID.
* Product.
* Category.
* Quantity.
* Amount.
* Purchase Date.

The objective is to group data by Category and Customer and identify top customers.

Python program:-

```
import pandas as pd.
df = pd.read_csv("supermarket.csv")
Category_sales = df.groupby("Category").agg(
    Total_quantity = ("Quantity","sum"),
    Total_Amount = ("Amount","sum"))
print("category-wise sales:")
print(Category_sales)
Customer_sales = df.groupby("Customer_ID").agg(
    Total_quantity = ("Quantity","sum"),
    Total_Amount = ("Amount","sum")
).sort_values("Total_Amount", ascending=False)
print("\nCustomer-wise sales:")
print(Customer_sales)
top_customers = Customer_sales.head(5)
print("\nTop 5 customers:")
print(top_customers)
```

Explanation:-

1) Category - wise analysis :-

df.groupby ("Category")["Amount"].sum()

2) Customer - wise analysis :-

df.groupby ("Total Amount", "Customer_ID").sum().

3) Finding top customers :-

.sort_values (("Total Amount"), ascending = False)

.head(5).

Conclusion:-

Pandas groupby() helps analyze purchasing behaviour by category and customer. It can identify high-performing categories, total quantities sold, total sales and top customers.